CS315 · ALGORITHMS ANALYSIS & DESIGN

Theory Review

The important theory for the midterm in one file

Muharib

What's inside

01 Midterm true/false and MCQ

02 L1: analysis of algorithms

03 L2: divide and conquer

04 L3: hash tables

05 L4: dynamic programming

06 L5: greedy

07 Check yourself

01 Midterm true/false and MCQ

Statement	Answer	Midterm
The code length in Huffman codes depends on the frequency of occurrence of characters.	**T**	451 462 472
Greedy algorithm can solve all the dynamic programming problems.	**F**	451 462 472
$T(n) = T(\frac{n}{2}) + T(\frac{n}{2}) + T(\frac{MERGE}{2})$ for MERGESORT.	**F**	451 462 472
An algorithm which uses the past results to find new results is ...	**DP**	451 462 472
The recursive activity-selection algorithm takes O(n log n).	**Other value: Θ(n)**	451 462
To compare two algorithms, the same HW/SW must be used without any consideration of inputs.	**F**	442 472
Theoretical analysis evaluates speed independent of the HW/SW environment.	**T**	442 472
The total of primitive operations represents the worst case.	**T**	442
DP takes far less time than methods that don't use overlapping sub-problems.	**T**	462

f(n) is ω(g(n)) if f is asymptotically ... g(n).	**strictly greater**	462
Best case of linear search.	**O(1)**	462
Which function is O(log n)?	**binary search**	462
Two nested loops of n each.	**O(n²)**	462
An algorithm is a disorganized set of steps in an unspecified time.	**F**	461
Running time typically grows with the input size.	**T**	461
Insertion sort sorts n elements in time about n.	**F (n²)**	461
Main operations of a dictionary: search, insert, delete.	**T**	461
Many optimal solutions but only one feasible solution.	**F**	461
The log file is most effective when ...	**insertions frequent, searches rare**	461
First step of a DP algorithm.	**characterize the structure**	461
Running time of Huffman encoding.	**O(n log n)**	461

Midterm 461, "fractional knapsack in O(n)": the key is uncertain here. If the items are not sorted, sorting takes `O(n log n)`, so the closer answer is F.

02 L1: analysis of algorithms

Term	Meaning
Algorithm	A step-by-step procedure for solving a problem in a finite amount of time.
Worst case	The one we analyze: easier to analyze, crucial for games, finance and robotics.
Limits of experiments	Must implement it; results may not cover other inputs; same HW/SW and same inputs needed.
Theoretical analysis	High-level description, a function of n, all inputs, independent of HW/SW.
RAM model	One CPU, unbounded numbered memory, unit-time access to any cell.
Primitive operations	Evaluate, assign, index, call, return. Each takes constant time.
arrayMax	Worst `7n − 2`, best `5n`, with the Temp line `7n`.
Changing hardware	Changes the constant only, not the growth rate.
prefixAverages	The first is `O(n²)`; the second (running sum) is `O(n)`.

Notation	Meaning
O	less than or equal (upper bound)
Ω	greater than or equal (lower bound)
Θ	equal
o	strictly less
ω	strictly greater

f(n) is O(g(n)) if there are positive constants c and n_0 such that $f(n) \leq c \cdot g(n)$ for $n \geq n_0$.
Use the smallest class and the simplest expression: 2n is O(n), 3n + 5 is O(n).

03 L2: divide and conquer

Point	What to remember
Steps	Divide → Conquer → Combine
Loop invariant	Initialization · Maintenance · Termination
Insertion sort	Best $O(n)$ (sorted), worst $O(n^2)$ (reversed). A comparison costs more than a swap.
Merge sort	$O(n \log n)$ in every case. Needs an auxiliary array and a merge step.
Recurrence	$T(n) = T(\frac{n}{2}) + T(\frac{n}{2}) + T(MERGE)$
Recursion tree	$\log(n) + 1$ levels; level t has 2^t problems of size $\frac{n}{2^t}$; each level's work is linear.
Quick sort	Splits around a pivot (maybe unequal), no auxiliary array or merge. Worst $O(n^2)$.

04 L3: hash tables

Point	What to remember
Log file	insert `O(1)`, find and remove `O(n)`. Good when insertions are frequent and searches rare.
Hash function	$h(x) = h_2(h_1(x))$: hash code map, then compression map.
When is it "good"?	It disperses keys randomly and evenly so collisions are few, and it is fast.
Hash code maps	Memory address, integer cast, component sum, polynomial accumulation (for strings).
Compression	Division $y \bmod N$ (N prime); MAD $(ay + b) \bmod N$ with $a \bmod N \neq 0$.
Chaining	Each cell holds a linked list. Simple, but needs memory outside the table.
Linear probing	Open addressing, the next cell. Weakness: items lump together.
Double hashing	$(i + j \cdot d(k)) \bmod N$, $d(k) = q - (k \bmod q)$, d is never zero, N is prime.
AVAILABLE	Marks a removed item so a search does not stop wrongly.
Performance	Expected $O(1)$, worst $O(n)$ (all keys collide). $\alpha = \frac{N}{M}$, expected probes $\frac{1}{1 - \alpha}$. Rehashing doubles the table.

05 L4: dynamic programming

Point	What to remember
Idea	Solve small sub-problems, store results, never recompute. Bottom-up. Bellman: every optimal policy consists of optimal sub-policies.
Two properties	Optimal sub-structure · Overlapping sub-problems
Versus D&C	In D&C sub-problems are independent and do not repeat.
Four steps	1. Characterize 2. Recursively define 3. Compute (bottom-up) 4. Construct
Applications	Matrix chain · LCS · Optimal BST
Fibonacci	Recursive $O(2^n)$, DP $O(n)$.
Matrix chain	Cost $p \cdot q \cdot r$. Goal: fewest multiplications. $O(n^3)$.
LCS	Match: diagonal + 1. Else: max(top, left). Length in the last cell. $O(mn)$.

06 L5: greedy

Point	What to remember
Idea	Takes the best option now and never reverses it. Top-down. Not always optimal.
Two properties	Greedy-choice property · Optimal substructure
Activity selection	Sort by finish time and take the one that finishes first. `Θ(n)` if sorted.
Fractional knapsack	Greedy is optimal: sort by $\frac{v}{w}$. `O(n log n)` due to sorting.
0-1 knapsack	Greedy may fail (empty space left); DP is needed.
Huffman	Join the two smallest frequencies each time. Left 0, right 1. More frequent = shorter code. Prefix code. `O(n log n)`.

07 Check yourself

1. Merge sort runs in $O(n^2)$ in the worst case.
2. In double hashing, N should be a prime.
3. Chaining stores colliding items in other cells of the table.
4. Greedy always gives the optimal solution for the 0-1 knapsack.
5. In DP, sub-problems are independent and never repeat.
6. Big-Omega gives a lower bound.
7. Insertion sort on a sorted array runs in O(n).
8. In Huffman coding, a rare character gets a shorter code.

#	Answer	Reason
1	**F**	`O(n log n)` in every case.
2	**T**	
3	**F**	That is open addressing. Chaining uses a linked list.
4	**F**	0-1 needs DP.
5	**F**	That is D&C. In DP they overlap.
8	**T**	
9	**T**	
10	**F**	The more frequent one gets the shorter code.